

Aarhus University

Project number - 2026F25

Player Controller System

Students:

Thea Teresa Lindgaard (202204350)
Niels Jakob Østerberg (202105879)
Kasper Stecher (202100451)
Kasper Bentsen (202103249)

Counsellor:

Lukas Esterle

Aarhus, May 29th 2026

Contents

1. Introduction	1
1.1. Requirements	2
2. Design	3
2.1. Architecture	3
2.2. Component pattern	4
2.3. State-Based Behavior	5
2.4. Observer Pattern	5
2.5. Input Design	6
2.6. Interface-Based Design	7
2.7. Movement Design	7
2.8. Ability Design	8
2.8.1. Interaction Design	8
2.9. Audio and Communication Design	9
2.10. Multiplayer Design	10
2.11. Camera Design	10
2.12. Handling Asymmetric Gameplay	11
3. Implementation	13
3.1. Player Object Structure	13
3.2. Input Handling	14
3.3. Movement Implementation	15
3.4. Ability Implementation	18
3.5. Interaction Implementation	19
3.6. Multiplayer Synchronization	21
3.7. Camera Binding and Behavior	22
3.8. Audio and Feedback Implementation	25
4. Testing Execution	27
5. Conclusion	28
Bibliography	29

1. Introduction

Author: Kasper Stecher

The purpose of the Player Controller System (PCS) is to provide the player character with the functionality and responsiveness required for the intended gameplay experience, including everything from movement and interaction to character-specific abilities.

The PCS contains the systems, settings, and gameplay logic that define how the player interacts with the game world, how movement is handled, and how the player character behaves during gameplay. As player interaction and movement are central elements of the game experience, the PCS serves as one of the core gameplay systems within the project.

The functionality of the Player Controller System is implemented through a collection of interconnected components responsible for movement, interaction, abilities, audio feedback, camera behavior, and overall player control.

The system follows a modular and component-based structure, where separate components handle specific responsibilities such as movement, interaction detection, camera control, visual feedback, and character abilities. These systems remain loosely coupled through shared interfaces and gameplay logic, improving maintainability, scalability, and flexibility throughout development.

1.1. Requirements

The requirements for the Player Controller System, as outlined in **Appendix 1.c.**, define the core functionality and usability goals necessary to support the intended gameplay experience.

Functional Requirements

- 1) Player Movement (**E1-US1**)
- 2) Player (Cat) Jumping (**E1-US2**)
- 3) Player Collision (**E1-US3**)
- 4) Player Camera (**E1-US4**)
- 5) Pick up items (**E2-US2**)
- 6) Push/pull Obstacles (**E2-US3**)
- 7) In-Game Communication (**E2-US5**)

The functional requirements focus on the systems required for the player to navigate and interact with the game world. These requirements form the foundation of the gameplay experience and ensure that both player characters are able to perform the actions necessary to complete levels and cooperate effectively.

Non-Functional Requirements

- 1) The game is fully navigable using a keyboard and mouse (**Usability - Must**)
- 2) The game is fully navigable using a controller (**Usability - Could**)

The non-functional requirements focus primarily on usability and accessibility. The system was designed to support responsive and intuitive controls across multiple input methods, ensuring that players are able to interact with the game comfortably regardless of their preferred control scheme.

2. Design

The purpose of the Player Controller System (PCS) is to provide the player character with the movement, interaction, and gameplay functionality required to support the intended gameplay experience in Parallax.

The PCS is designed as a modular collection of interconnected gameplay systems responsible for movement, interaction, camera control, character abilities, and player feedback. These systems work together to create responsive and intuitive player behavior throughout gameplay.

As Parallax is built around asymmetric cooperative gameplay, the PCS must support multiple player roles with unique gameplay mechanics while still maintaining a shared gameplay framework and a consistent player experience.

The system is implemented using Unity's component-based architecture and follows a modular design structure, where individual components encapsulate specific responsibilities while communicating through shared gameplay logic and interfaces. This approach improves maintainability, scalability, and flexibility throughout development.

2.1. Architecture

The PCS follows Unity's component-based architecture, where gameplay functionality is distributed across multiple specialized components attached to the player object.

Rather than implementing all functionality within a single monolithic controller, the system separates responsibilities into smaller interconnected components with clearly defined purposes. This separation allows individual systems to be developed, tested, and modified independently without heavily impacting other parts of the controller architecture.

The PCS consists of multiple systems responsible for:

- movement
- input handling
- interaction
- camera control
- abilities
- audio feedback
- role handling
- visual feedback
- multiplayer synchronization

Communication between systems is handled through shared gameplay logic, interfaces, Unity component references, and event-driven behavior. This allows the systems to remain loosely coupled while still functioning together as part of the overall player experience.

The modular architecture improves maintainability, scalability, flexibility, and readability throughout development, while also making it easier to extend the system with additional gameplay mechanics and role-specific behavior.

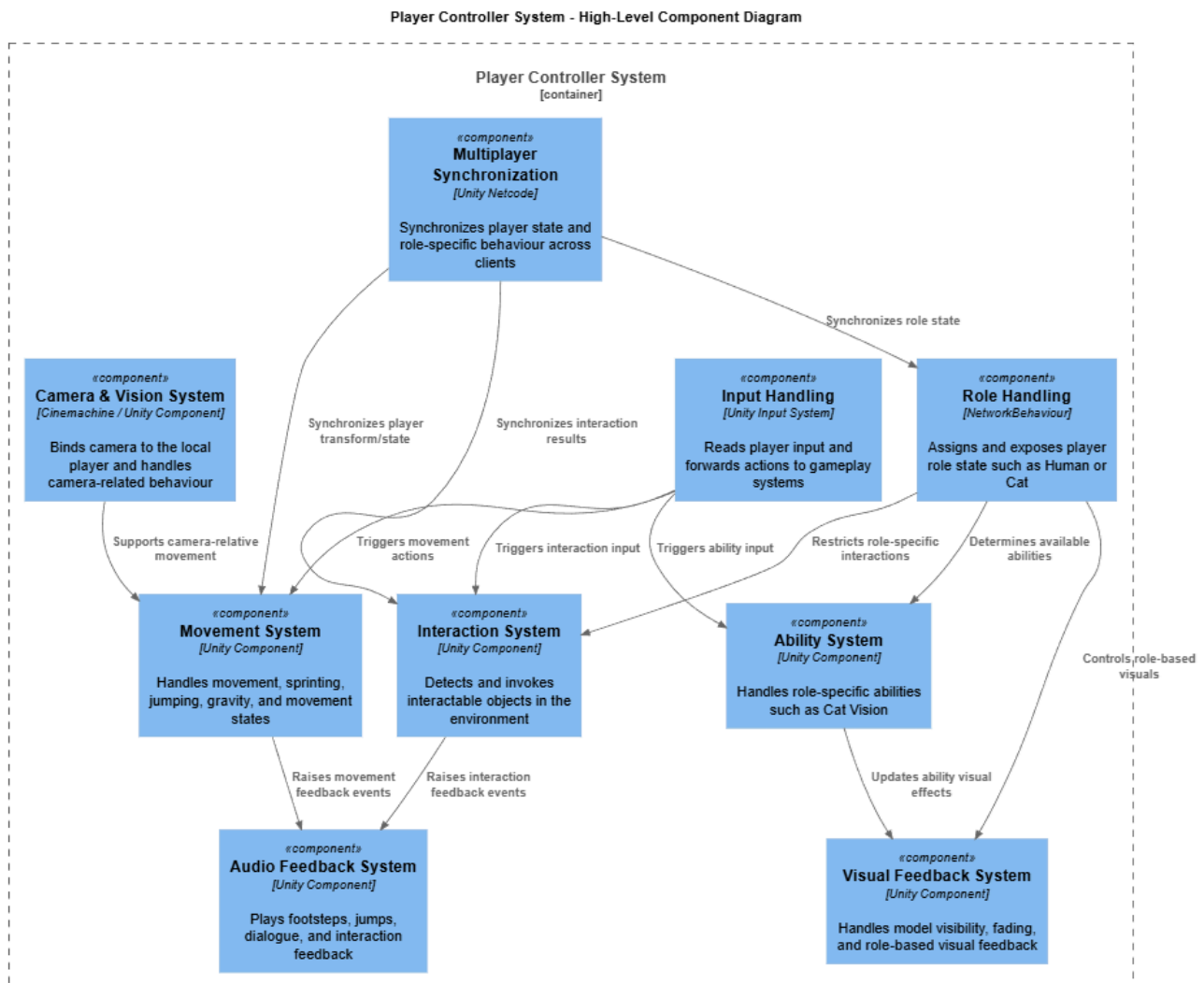


Figure 1: High level diagram of the Player Controller system (PCS).

2.2. Component pattern

The PCS heavily relies on Unity's component-based architecture.

Individual gameplay features are implemented as separate components attached to the player object.

Examples include:

- PlayerMovementController
- PlayerMovement
- PlayerJumpAbility
- PlayerCatVision
- PlayerCatVisionInput
- PlayerInteraction / Interactor

- InteractionPromptUI
- PlayerAudioController
- RoleController

This allows gameplay systems to remain modular and reusable while reducing coupling between systems.

2.3. State-Based Behavior

The PCS utilizes state-based behavior to control how player systems respond under different gameplay conditions. Player movement, interaction, and camera functionality can change dynamically depending on the current active gameplay conditions, allowing the controller to adapt behavior while maintaining predictable and consistent gameplay logic.

Examples of gameplay-related states include:

- sprinting
- carrying objects
- movement locking
- CatVision activation

These states influence gameplay by modifying movement speed, available actions, camera behavior, or interaction capabilities. Rather than relying on separate controller implementations, the PCS manages these behavior changes through modular components, conditional logic, and shared gameplay systems.

2.4. Observer Pattern

The Observer pattern is utilized throughout the PCS using Unity events, Input System callbacks, and network variable updates. Systems subscribe to gameplay events and react when state changes occur, allowing communication between systems without creating tight coupling between components.

Examples include:

- interaction prompts reacting to interactable detection
- network variables notifying systems of role changes
- input callbacks triggering gameplay actions
- audio systems reacting to movement or jump events

This event-driven approach allows gameplay systems to remain modular and loosely connected while still responding dynamically to gameplay changes.

2.5. Input Design

Input handling is separated from gameplay logic using Unity's Input System. This allows gameplay systems to remain independent from specific input devices and supports both keyboard/mouse and controller input configurations.

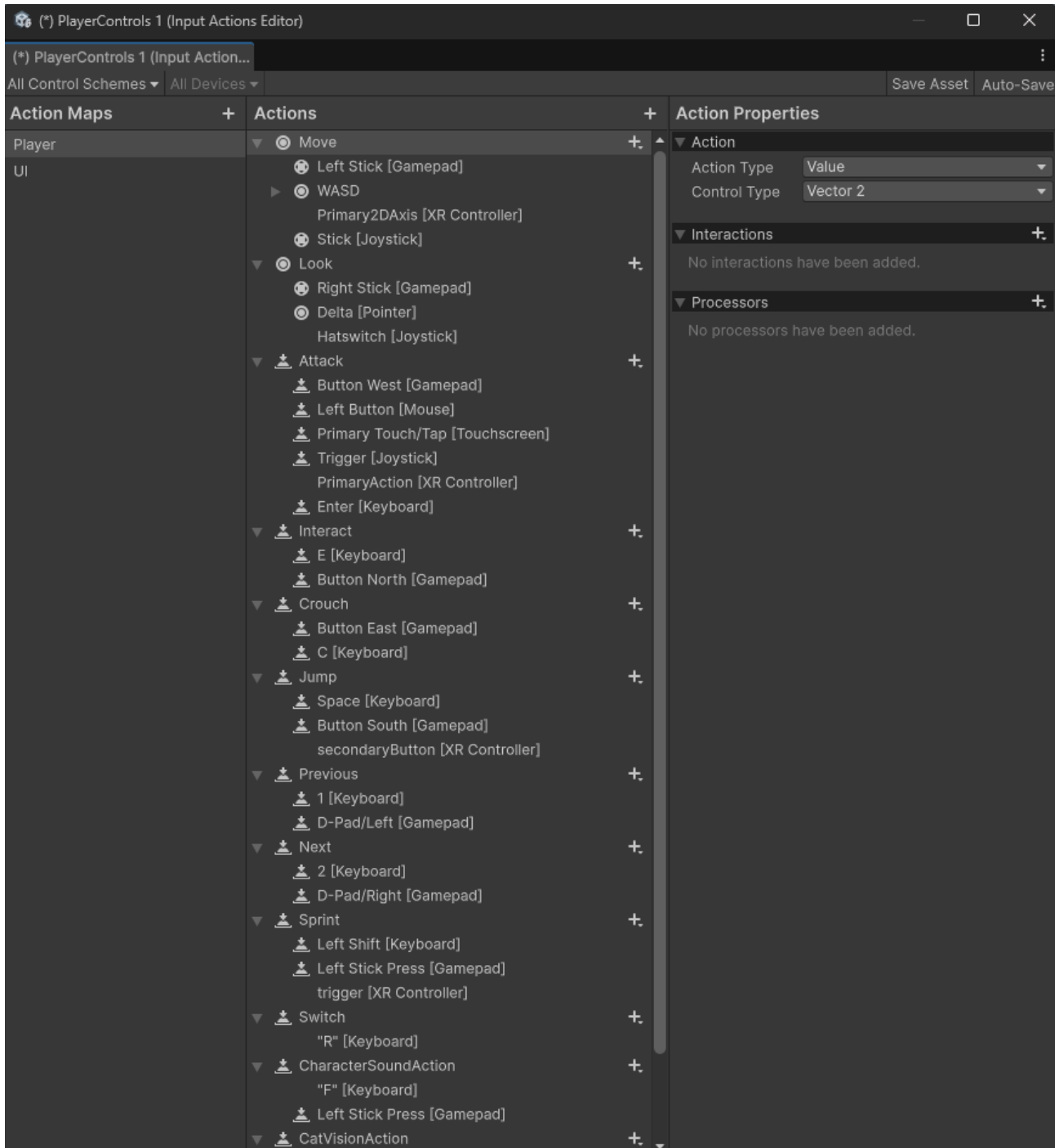


Figure 2: Screenshot of the Input Action Map used in Parallax, showing how gameplay actions are mapped to different input bindings.

Actions were defined in an Input Action Map rather than being hardcoded directly into gameplay scripts. This reduces coupling between input handling and gameplay logic, making the system easier to maintain and less dependent on individual scripts being modified during development.

2.6. Interface-Based Design

The PCS utilizes interface-based design to define shared gameplay behavior across different systems and gameplay objects.

Interfaces are used to standardize gameplay functionality while reducing dependencies between individual implementations. This allows systems to communicate through shared behavioral contracts rather than relying on specific concrete classes.

Examples include:

- IInteractable
- IActivatable
- IJump
- ISprint

```
public interface IJump
{
    void RequestJump();
    void ReleaseJump();
}
```

Code Snippet 1: Example implementation of the IJump interface.

Using interfaces enables gameplay systems to remain modular, reusable, and extensible throughout development. It also simplifies the integration of new mechanics and gameplay objects, as systems can interact with any object implementing the required interface without needing knowledge of its internal implementation.

2.7. Movement Design

The movement system was designed to provide responsive and intuitive player control while supporting the cooperative gameplay of Parallax. Movement needed to feel smooth and predictable, as precise positioning and timing are important aspects of both puzzle solving and environmental traversal.

Player movement is implemented using Unity's CharacterController **[1]** component rather than a physics-driven Rigidbody approach. This decision provided more direct control over movement, collision handling, and traversal, while also improving responsiveness.

The movement architecture is separated into multiple specialized components. PlayerMovementController acts as the coordinating layer between the Unity Input System **Figure 2** and the movement-related systems, while PlayerMovement handles movement logic such as walking, sprinting, gravity, and grounded detection. Jumping functionality is implemented separately through Player-

JumpAbility, allowing movement abilities to be enabled or restricted depending on the player's assigned role.

Movement is calculated relative to the camera direction, ensuring that player input remains aligned with the current viewing perspective. This improves usability and allows navigation within the 3D environment to feel more natural regardless of camera orientation.

The movement system also includes directional strafing and dynamic rotation to improve movement readability and overall gameplay feel. Rotation behavior changes depending on movement direction, camera orientation, and player input, helping create smoother transitions.

Additional traversal mechanics such as sprinting and jumping were implemented to support gameplay variation and asymmetric level design. Sprinting allows players to increase movement speed, while jumping enables the Cat player to reach elevated areas and traverse obstacles unavailable to the Human player.

Grounded detection and gravity handling are integrated into the movement systems to ensure stable movement and reliable jump execution throughout gameplay.

2.8. Ability Design

Gameplay abilities such as jumping, sprinting, and CatVision were implemented as separate components in order to maintain modularity and reduce coupling between gameplay systems. Separating abilities into individual components also supports SOLID [2] design principles by ensuring that each component is responsible for a specific gameplay ability.

As Parallax is designed around asymmetric cooperative gameplay, the Human and Cat roles were intentionally given different gameplay abilities. The RoleController system is responsible for tracking the currently assigned player role and is used by gameplay systems to determine which abilities and interactions are available to the player.

This architecture allows role-specific abilities to be enabled, disabled, or extended without requiring major changes to the underlying controller framework.

2.8.1. Interaction Design

The interaction system was designed to provide a framework for player interaction throughout the game. As many gameplay mechanics in Parallax rely on environmental interaction and cooperation between players, the system needed to support multiple object types.

Interaction detection is handled using raycasts and spherecasts projected from the player toward nearby objects. Raycasts allow the system to detect objects directly in front of the player, while spherecasts provide a more forgiving interaction area, reducing the need for precise aiming and improving usability during gameplay.

Interactable gameplay objects implement a shared interaction interface, allowing different object types to be handled through a common interaction system. This includes objects such as:

- buttons
- doors
- keys
- movable boxes

Using a shared interface-based structure allows new interactable objects to be added without requiring major changes to the core interaction system. Interfaces were chosen over inheritance to improve extensibility and reduce coupling between gameplay systems. This approach allows new gameplay objects and mechanics to integrate with the existing interaction framework by implementing the required behavioral contracts. This was particularly important during development, as gameplay mechanics and interactable object types changed frequently throughout prototyping and iteration.

The PCS also includes an interaction prompt system that provides visual feedback when the player is able to interact with an object. Prompts display contextual interaction messages, helping communicate available actions to the player while reducing the need for explicit tutorials.

Many interactions are tied directly to cooperative gameplay mechanics. Certain interactions may only be available to a specific player role, requiring cooperation between the Human and Cat players in order to progress through levels and solve puzzles.

2.9. Audio and Communication Design

As Parallax is designed around cooperative asymmetric gameplay, communication between players plays an important role throughout gameplay. In addition to external player communication, the PCS also includes in-game audio feedback systems used to support player awareness and cooperative interaction.

Role-specific character sounds were implemented to provide feedback during gameplay actions and to reinforce the distinction between the Human and Cat roles. These sounds can be triggered through gameplay events such as movement, jumping, interaction, or player-controlled audio actions.

The audio system utilizes an event-driven structure, where gameplay systems trigger audio events that are handled independently by dedicated audio components. This reduces coupling between gameplay logic and audio playback while improving modularity and maintainability.

The use of audio feedback also helps communicate gameplay information to other players during cooperative puzzle solving and traversal, supporting player coordination without relying entirely on visual feedback or external voice communication.

2.10. Multiplayer Design

As Parallax is designed as a cooperative multiplayer experience, the Player Controller System was built to support synchronized gameplay across multiple connected clients. Multiplayer functionality was implemented using Unity Netcode for GameObjects (NGO).

The PCS uses NGO's ownership system to ensure that player input and movement are only processed by the owning client. This prevents multiple clients from controlling the same player object while still allowing gameplay state and visual updates to remain synchronized across the network.

Network communication is handled through Remote Procedure Calls (RPCs) and `NetworkVariables`. RPCs are used for gameplay events and interactions that must be executed across the network, such as activating buttons, interacting with objects, or triggering cooperative gameplay mechanics.

`NetworkVariables` are used to synchronize persistent gameplay states between clients. This includes systems such as role handling, interaction states, and activation states for gameplay objects. By synchronizing important gameplay data through `NetworkVariables`, all connected players maintain a consistent view of the game world.

The multiplayer synchronization strategy focuses on keeping gameplay interactions authoritative while minimizing unnecessary network traffic. Systems are designed to synchronize only the data required for gameplay consistency, while local systems such as camera handling and player input remain client-side.

Additional considerations were made for scene transitions, player spawning, and camera ownership to ensure that each player maintains correct control and perspective throughout gameplay sessions.

2.11. Camera Design

The camera system was designed to support player awareness, movement readability, and environmental navigation throughout gameplay. As Parallax is played from a third-person perspective, the camera plays an important role in both traversal and cooperative gameplay communication.

Camera behavior was implemented using Unity Cinemachine, allowing camera movement and tracking to remain smooth and responsive while reducing the need for complex custom camera logic. Cinemachine also provided flexibility for adjusting camera behavior based on gameplay situations and player abilities.

The PCS uses local player camera binding to ensure that each connected player only controls and views their own character during multiplayer gameplay. Camera targets are assigned dynamically at runtime, allowing the camera system to correctly follow the locally owned player object after spawning and scene transitions.

Additional camera collision and obstruction handling was implemented using Cinemachine's built-in camera collision system. By assigning specific collision layers to environmental objects, it was possible to control which objects were considered significant enough to trigger camera adjustment. This prevented environmental geometry such as walls and large objects from obstructing the player character during gameplay. When obstructions occurred, the camera dynamically adjusted its position and zoom distance to maintain visibility of the player while preserving smooth camera movement.

Special camera behavior was also implemented for gameplay abilities such as CatVision. When CatVision is activated, the camera adjusts to better support the ability by modifying elements such as zoom and visual focus. These camera adjustments help reinforce gameplay feedback while improving visibility of important environmental information during puzzle solving and traversal.

2.12. Handling Asymmetric Gameplay

Parallax is designed around asymmetric cooperative gameplay, where each player has different gameplay capabilities and perspectives.

The PCS therefore supports role-specific gameplay behavior through the RoleController system.

Each player is assigned a gameplay role:

- Human
- Cat

Depending on the assigned role, different gameplay systems, abilities, interactions, and role-specific gameplay settings become available.

Examples include:

- the Cat being able to jump and use CatVision
- the Human being able to interact with heavier objects

Although gameplay capabilities differ between roles, both players share the same underlying controller framework. Role-specific movement values and gameplay behavior are configured through the PlayerRoleStats system, allowing each role to maintain unique movement characteristics while still using the same modular controller architecture.

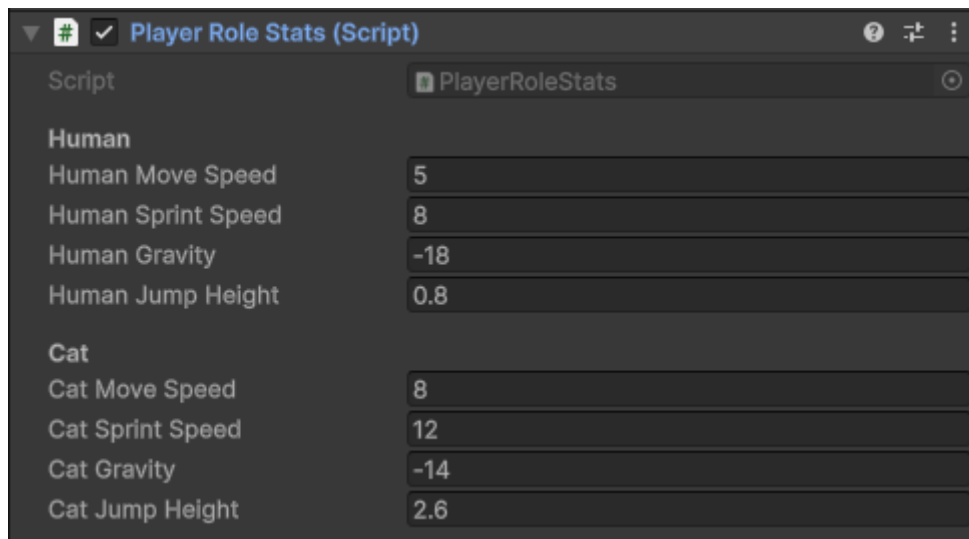


Figure 3: PlayerRoleStats component on the BasePlayerCharacter prefab, showing role-specific movement configuration values.

For example, the Cat is able to move faster and jump higher than the Human. These values are configured on the BasePlayerCharacter prefab and were adjusted throughout development to help achieve the desired movement “feel” while also balancing the difficulty of certain gameplay manoeuvres.

3. Implementation

The Player Controller System was implemented as a collection of Unity components attached to the BasePlayerCharacter prefab. Each component is responsible for a specific part of player functionality, such as input handling, movement, interaction, abilities, audio feedback, or role handling.

This implementation follows the modular structure described in the design section, where gameplay behavior is divided into smaller components rather than being handled by a single player controller script.

3.1. Player Object Structure

The Player Controller System was implemented using a modular component-based structure attached to the BasePlayerCharacter prefab. Rather than relying on a single monolithic player controller, gameplay functionality was separated into multiple specialized components responsible for different gameplay systems.

Each component handles a specific responsibility within the overall controller architecture. Examples include:

- movement handling
- input processing
- interaction detection
- gameplay abilities
- camera behavior
- audio feedback
- role handling
- multiplayer synchronization

This structure allowed systems to remain loosely coupled while still communicating through shared gameplay logic, interfaces, Unity events, and component references.

Core gameplay systems such as movement and interaction were separated into dedicated scripts including:

- PlayerMovement
- PlayerMovementController
- PlayerJumpAbility
- Interactor
- RoleController
- PlayerCatVision
- PlayerAudioEvents

The modular structure improved scalability throughout development, as new gameplay systems and abilities could be added to the player object without requiring major architectural changes to the existing controller framework.

3.2. Input Handling

Player input was implemented using Unity's Input System in order to support multiple input devices while keeping gameplay systems independent from specific hardware configurations.

Input handling was separated from gameplay logic through the `PlayerMovementController` component. This component acts as an intermediary layer between the Unity Input System and the gameplay systems responsible for movement, interaction, abilities, and camera behavior.

Input actions were configured through Unity Input Action Maps, where gameplay actions such as movement, jumping, sprinting, interaction, and camera control were mapped independently of specific devices. This allowed the PCS to support both keyboard and mouse input as well as controller input without requiring major changes to gameplay systems.

Rather than allowing gameplay systems to directly process raw input data, the controller component translates input actions into gameplay behavior by communicating with specialized gameplay systems such as:

- `PlayerMovement`
- `PlayerJumpAbility`
- `Interactor`
- `PlayerCatVision`

In multiplayer gameplay, input processing is restricted to the locally owned player object through NGO ownership checks. This ensures that only the owning client can control a player character while gameplay state remains synchronized across connected clients.

3.3. Movement Implementation

Player movement was implemented using Unity's CharacterController [1] component rather than a physics-driven Rigidbody approach. This provided more direct control over player movement, collision handling, gravity, and traversal behavior while improving responsiveness and gameplay consistency.

Early movement prototypes and controller experiments were partially inspired by community-created Unity controller tutorials and examples [3].

Movement functionality was separated into multiple specialized components in order to maintain modularity throughout the controller architecture.

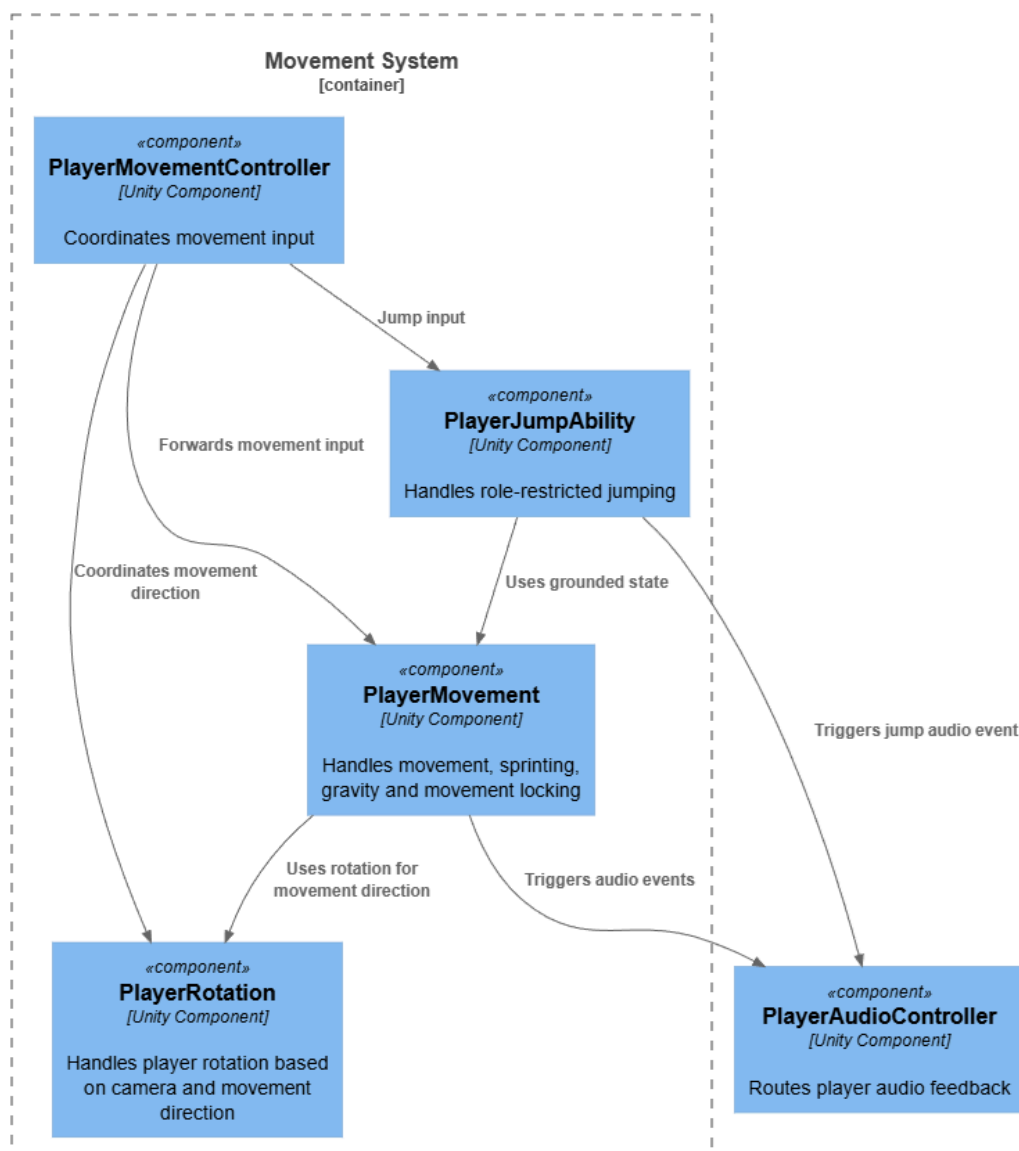


Figure 4: Component diagram of the Movement System within the Player Controller System (PCS), showing how movement input is coordinated between movement, rotation, jumping, and audio-related gameplay components.

The `PlayerMovementController` component is responsible for receiving player input and coordinating communication between the movement-related systems. Movement calculations and traversal behavior are primarily handled by the `PlayerMovement` component, which manages:

- walking
- sprinting
- gravity
- grounded detection
- directional movement
- character rotation

Movement System - Class Diagram

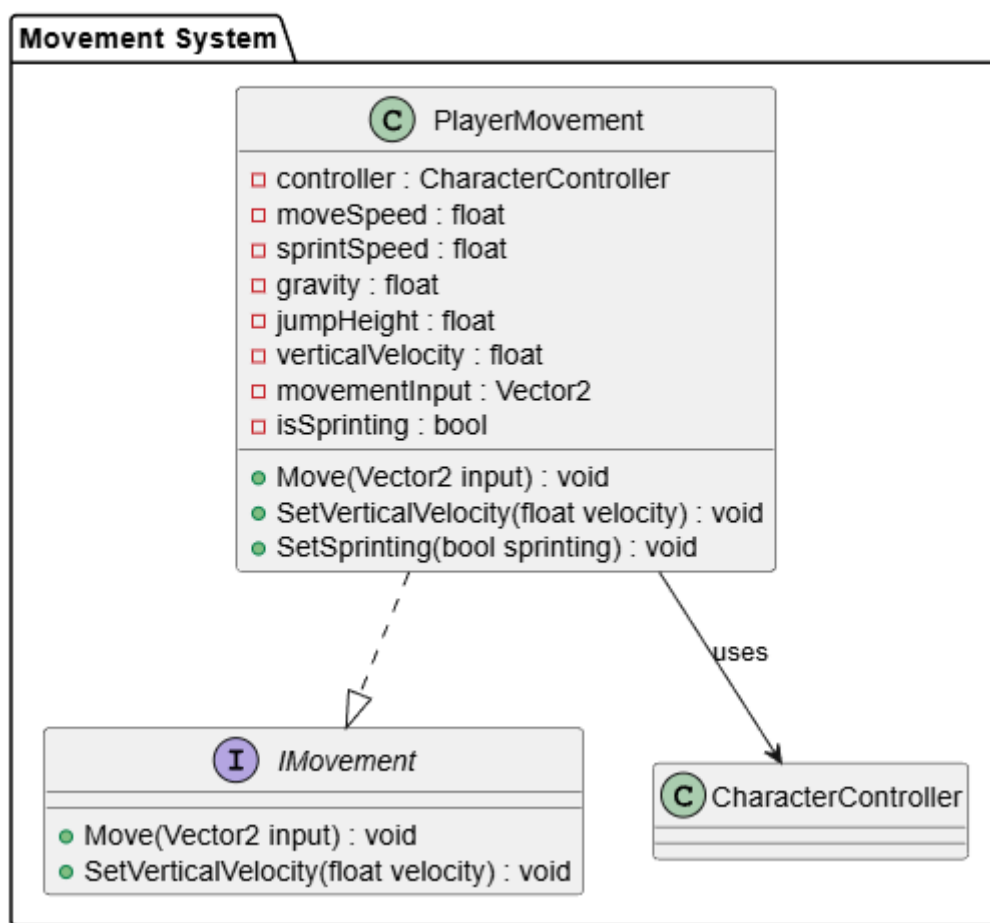


Figure 5: Class diagram of the Movement System within the Player Controller System, showing how `PlayerMovement` implements the `IMovement` interface and uses Unity's `CharacterController` to move the player.

Jumping functionality was implemented separately through the `PlayerJumpAbility` component. Separating jumping from the primary movement logic allowed movement abilities to be enabled or restricted depending on the currently assigned player role.

Movement direction is calculated relative to the current camera orientation, allowing player movement to remain aligned with the viewing perspective. This improves usability and creates more intuitive navigation within the 3D environment.

Additional movement such as strafing, dynamic character rotation, and sprinting were implemented to improve movement readability and gameplay feel. Rotation behavior changes dynamically depending on movement direction and camera orientation, helping create smoother transitions during traversal.

Grounded detection and gravity handling were implemented directly within the movement systems to ensure stable movement behavior and reliable jump execution during gameplay. Special attention was given to movement responsiveness and traversal consistency, as player movement forms a central part of both puzzle solving and cooperative gameplay throughout Parallax.

3.4. Ability Implementation

Player abilities were implemented as separate Unity components attached to the BasePlayerCharacter prefab. This allowed each ability to remain independent from the main movement and interaction systems while still being available as part of the overall Player Controller System.

```
if (!roleController.IsCat) return;
```

Code Snippet: Example of a role-based ability check.

Examples of implemented abilities include:

- PlayerJumpAbility
- PlayerCatVision
- sprinting behavior through PlayerMovement

The PlayerJumpAbility component handles jump-specific behavior such as jump requests, jump release, and vertical movement changes. By keeping jumping separate from the general movement logic, the system could more easily restrict or enable jumping depending on the player's assigned role.

CatVision was implemented as a role-specific ability for the Cat player. When activated, CatVision modifies gameplay feedback by revealing specific objects or visual indicators in the environment. This supports asymmetric puzzle solving by allowing the Cat player to perceive information that may not be available to the Human player.

Role-specific ability access is controlled through the RoleController system. Gameplay components use the assigned player role to determine whether an ability should be available. This made it possible to give the Human and Cat different gameplay capabilities while still using the same underlying player controller framework.

Implementing abilities as independent components improved modularity and made it easier to adjust, extend, or disable individual abilities during development without requiring major changes to the rest of the PCS.

3.5. Interaction Implementation

The interaction system was implemented through a combination of detection systems, shared interfaces, and modular interactable gameplay objects.

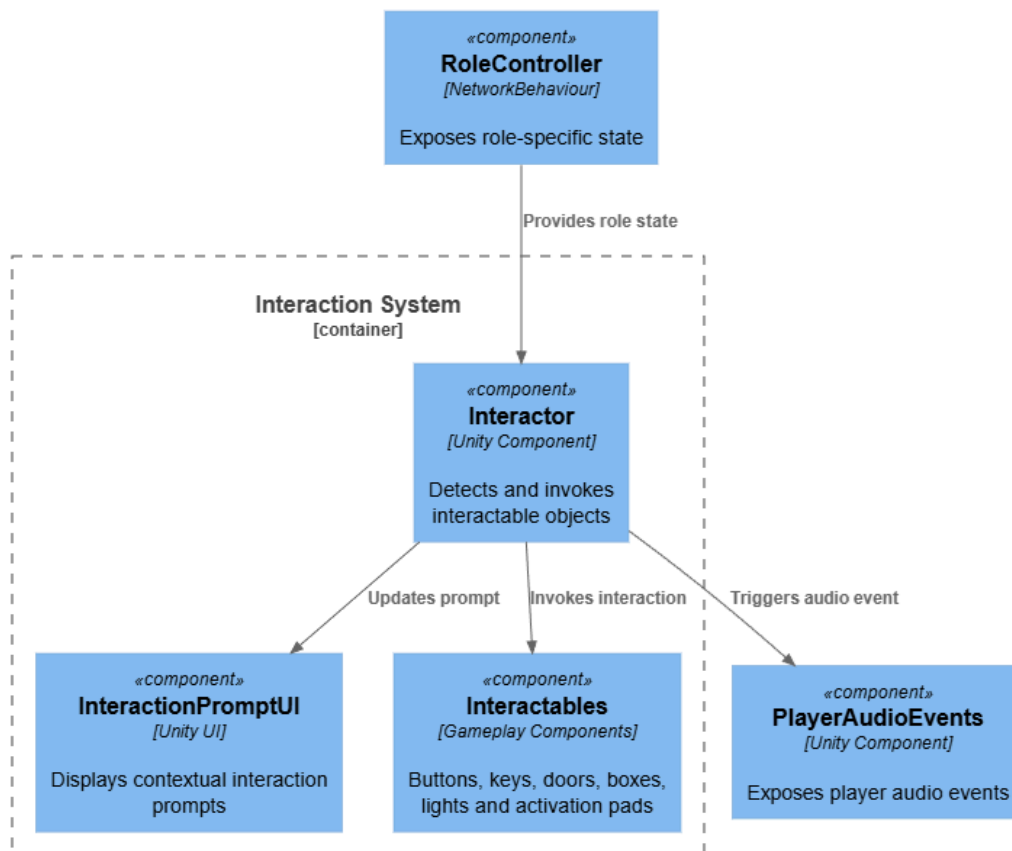


Figure 6: Component diagram of the Interaction System within the Player Controller System (PCS), illustrating how player interaction is coordinated between interactable objects, UI prompts, role handling, audio feedback, and interaction detection.

Player interaction detection is handled by the Interactor component, which performs raycasts and spherecasts from the player toward nearby objects. Raycasts provide precise forward-facing interaction detection, while spherecasts extend the interaction area to improve usability and reduce the need for exact player aim during gameplay.

```
if (hit.collider.TryGetComponent<IInteractable>(out var interactable))
{
    interactable.Interact(gameObject);
}
```

Code Snippet: Example implementation of the IJump interface.

Objects that can be interacted with implement the IInteractable interface. This allows different gameplay objects to be handled through the same interaction framework regardless of their specific internal implementation.

Examples of implemented interactable objects include:

- `ButtonInteractable`
- `RotateInteractable`
- `KeyInteractable`
- `SmallBoxCarryable`
- `LargeBoxDraggable`

Using shared interfaces allowed gameplay systems to interact with objects through common behavioral contracts rather than concrete implementations. This simplified the addition of new interactable gameplay objects during development while reducing dependencies between systems.

Additional interaction logic was implemented through condition-based systems such as:

- `IInteractCondition`
- `RequiresActivatedCondition`
- `ConditionalInteractable`

These systems made it possible to restrict interactions based on gameplay requirements such as activation states, player roles, or progression conditions.

Interaction feedback is provided through the `InteractionPromptUI` system, which displays contextual interaction prompts when the player is able to interact with an object. This helped communicate available gameplay actions while improving usability and reducing the need for explicit tutorials.

Many interactable systems were also integrated with multiplayer synchronization through `NGO` RPCs and `NetworkVariables`, ensuring that gameplay interactions remain synchronized across connected clients during cooperative gameplay.

3.6. Multiplayer Synchronization

Multiplayer synchronization was implemented using Unity Netcode for GameObjects (NGO). As Parallax is designed as a cooperative multiplayer experience, the PCS needed to ensure that player movement, role-specific behavior, interactions, and gameplay states remained consistent across connected clients.

Input and camera behavior are processed locally on the owning client. Ownership checks are used to ensure that only the player who owns a specific player object can control its movement, camera, and input-driven abilities. This prevents multiple clients from controlling the same character while still allowing relevant gameplay state to be synchronized across the network.

Gameplay-critical interactions are handled through server-authoritative logic. When a player interacts with an object, the request is sent to the server, which validates and applies the gameplay change before synchronizing the result to all connected clients. This approach helps avoid conflicting gameplay states between players.

```
if (!IsOwner) return;
```

Code Snippet: Example implementation of the IJump interface.

Network synchronization is primarily handled through:

- NetworkVariables
- RPCs
- ownership checks

NetworkVariables are used for persistent gameplay states that need to remain synchronized between clients. This includes role assignment, activation states, collected keys, and other gameplay conditions that must be visible to all players.

RPCs are used for gameplay events that need to be triggered across the network, such as activating buttons, opening doors, moving platforms, or updating interactable objects. This allowed cooperative puzzle mechanics to remain consistent for both players during gameplay.

Local-only systems, such as camera handling and input reading, were kept client-side to avoid unnecessary network traffic. Only gameplay state required for multiplayer consistency was synchronized, helping keep the system more efficient and easier to maintain.

This synchronization structure allowed the PCS to support cooperative multiplayer gameplay while maintaining clear separation between local player control and shared gameplay state.

3.7. Camera Binding and Behavior

The camera system was implemented using Unity Cinemachine in order to provide smooth third-person camera behavior while reducing the need for complex custom camera logic.

Camera functionality is handled through a combination of Cinemachine camera components and custom binding logic. During multiplayer gameplay, camera ownership and target assignment are handled dynamically to ensure that each player camera follows the correct locally owned player object.

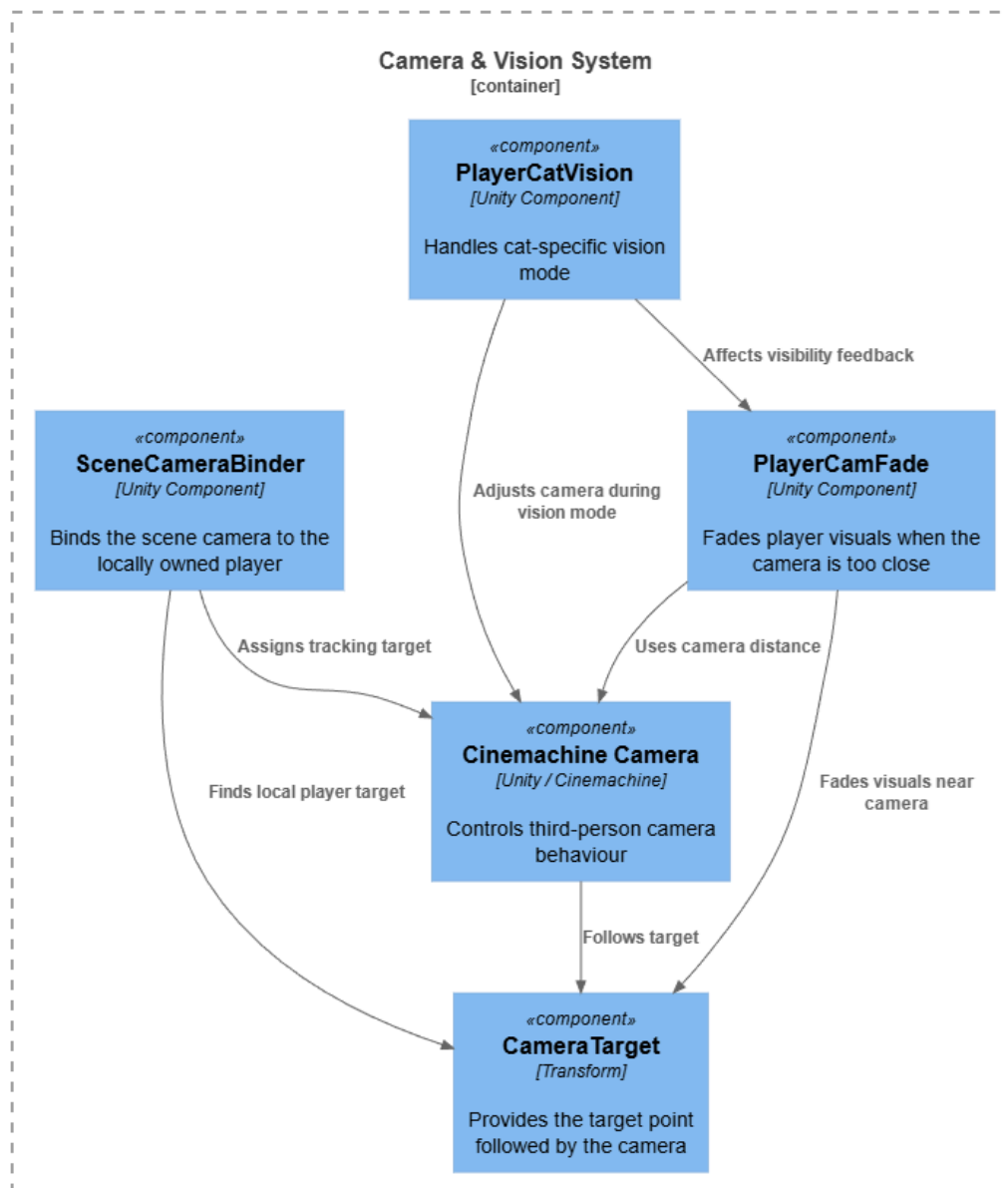


Figure 7: Component diagram of the Camera and Vision System within the Player Controller System (PCS), illustrating how camera binding, player visibility, camera targeting, and cat vision functionality are coordinated using Cinemachine and gameplay components.

The SceneCameraBinder system dynamically assigns the Cinemachine camera target at runtime during multiplayer gameplay. After a player object spawns through NGO, the system locates the locally owned player object and searches for its CameraTarget transform. This target is then assigned to the

Cinemachine camera's tracking target, ensuring that each client camera follows the correct player instance during gameplay and scene transitions. Performing the camera binding at runtime avoids issues where the camera could otherwise remain linked to the original prefab or an incorrect player object.

```
private void LateUpdate()
{
    if (bound || cam == null) return;

    if (NetworkManager.Singleton == null || !NetworkManager.Singleton.IsClient)
        return;

    var playerObject = NetworkManager.Singleton.LocalClient?.PlayerObject;
    if (playerObject == null)
        return;

    Transform target = playerObject.transform.Find("CameraTarget");
    if (target == null)
    {
        Debug.LogWarning("CameraTarget not found, using root");
        target = playerObject.transform;
    }

    cam.Target.TrackingTarget = target;

    bound = true;

    Debug.Log($"[SceneCameraBinder] Bound camera to {target.name} on {playerObject.name}", this);
}
```

Code Snippet: Example implementation of the SceneCameraBinder system, dynamically assigning the local player's CameraTarget to the Cinemachine camera during multiplayer runtime initialization.

Camera movement and rotation are synchronized with player movement to maintain responsive third-person traversal behavior. Movement direction is calculated relative to the camera orientation, allowing player navigation to remain intuitive regardless of viewing angle.

Additional camera behavior was implemented to support gameplay readability and environmental navigation. Cinemachine's built-in camera obstruction handling was used to reduce situations where walls or environmental objects block visibility of the player character. When obstructions occur, the camera dynamically adjusts its distance and position to preserve visibility while maintaining smooth movement behavior.

Role-specific camera behavior was also implemented for gameplay abilities such as `PlayerCatVision`. When `CatVision` is activated, the camera modifies visual behavior through zoom adjustments and environmental emphasis in order to improve visibility of important gameplay elements during puzzle solving and traversal.

Special attention was given to ensuring that camera behavior remained smooth and unobtrusive during gameplay, as camera responsiveness and visibility play a central role in both player movement and cooperative gameplay communication throughout `Parallax`.

3.8. Audio and Feedback Implementation

Audio and feedback functionality was implemented to support player awareness, gameplay responsiveness, and cooperative communication during gameplay.

The PCS uses an event-driven audio structure where gameplay systems trigger audio events instead of directly controlling audio playback. This reduces coupling between gameplay logic and audio behavior while keeping the system modular and easier to extend.

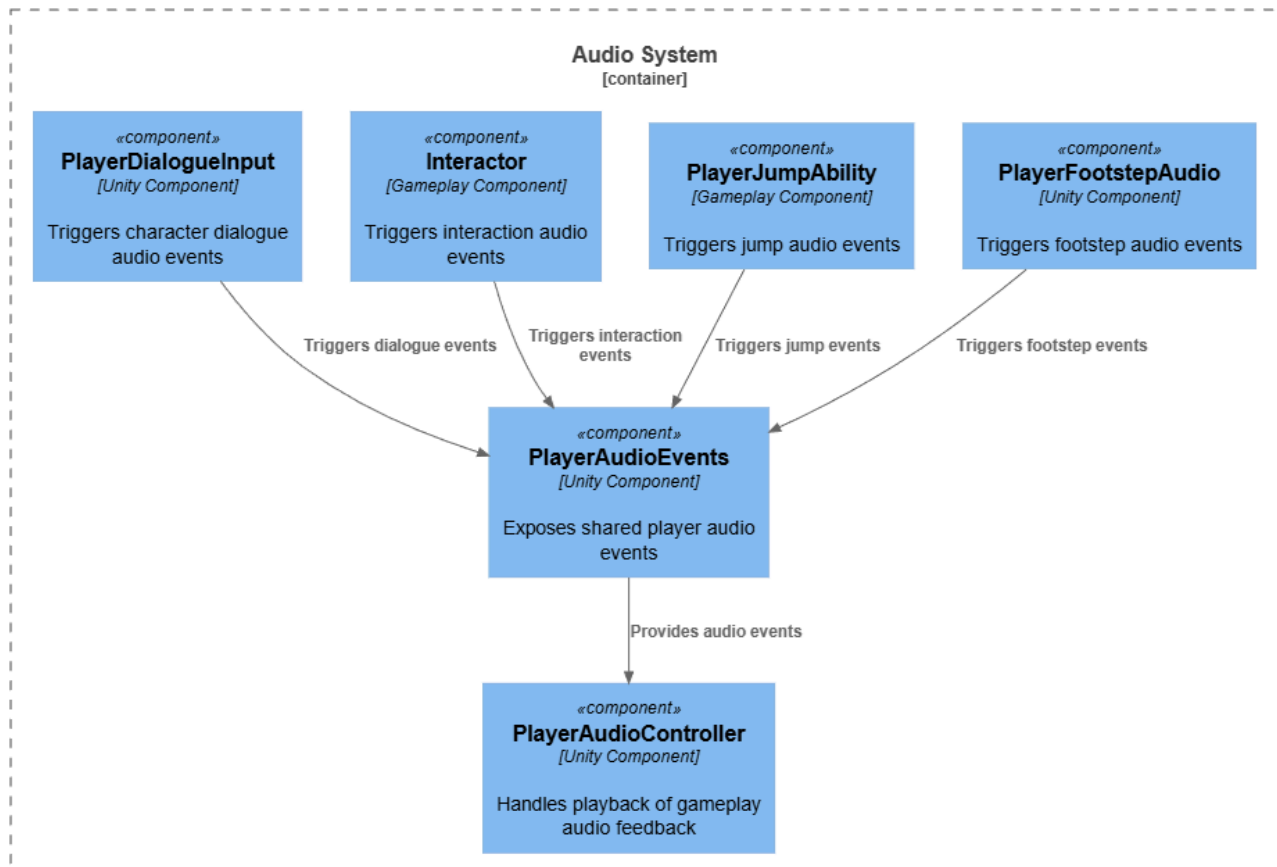


Figure 8: Component diagram of the Audio System within the Player Controller System (PCS), illustrating how gameplay systems trigger shared audio events that are routed through the centralized player audio controller.

The PlayerAudioEvents component defines shared gameplay audio events such as:

- jumping
- dialogue sounds
- footstep sounds

Audio System - Class Diagram

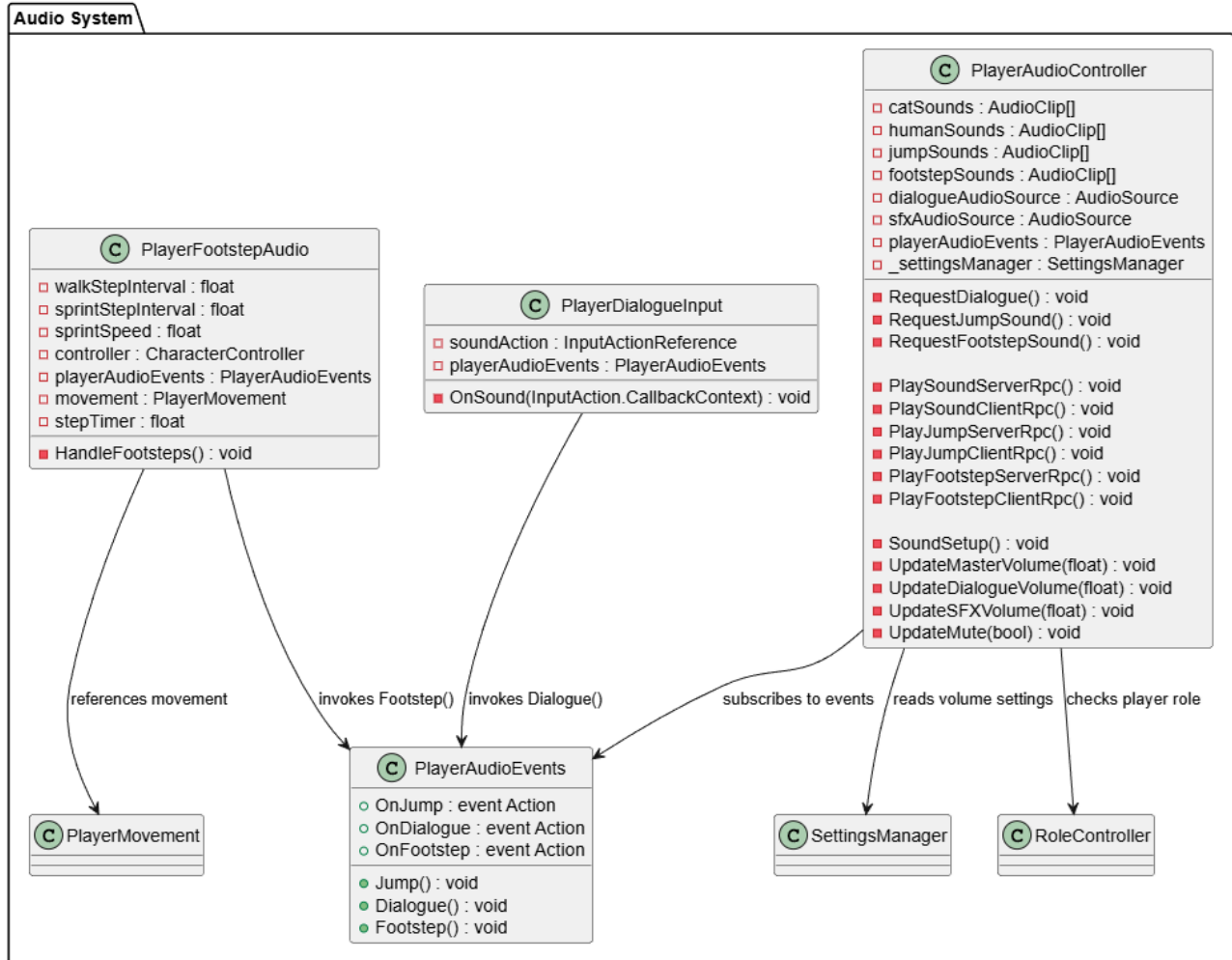


Figure 9: Component diagram of the Audio System within the Player Controller System, showing how gameplay systems trigger shared audio events that are routed through the centralized player audio controller.

These events are invoked by gameplay systems when relevant player actions occur. Dedicated audio components then listen for these events and handle the actual playback of sound effects or character sounds.

Role-specific audio feedback was implemented to reinforce the difference between the Human and Cat roles. For example, the Cat and Human can use different character sounds, helping players distinguish between roles and communicate actions during cooperative gameplay.

Footstep and movement-related audio feedback was also implemented to improve responsiveness and make player movement feel more grounded. This supports the overall player experience by making actions feel more immediate and connected to the game world.

By separating audio events from audio playback, the system remains flexible and reusable. New sound effects or feedback responses can be added without requiring major changes to the gameplay systems that trigger them.

4. Testing Execution

Testing of the PCS was primarily conducted through manual gameplay testing. Traditional unit testing was not heavily utilized, as many systems relied on real-time player interaction and in-game behavior that could not easily be validated through isolated automated tests.

Systems such as movement, interaction, jumping, camera behavior, and role-specific abilities required direct gameplay evaluation to determine whether the mechanics felt responsive, intuitive, and functioned correctly under different gameplay situations.

The testing approach therefore focused on continuous developer testing during implementation, where features were tested directly in-game as they were developed. Before new additions were merged into the staging branch, other group members reviewed and tested the implemented functionality to help identify bugs, gameplay issues, and unintended behavior.

5. Conclusion

The Player Controller System turned out to be a significantly larger entity than initially anticipated, as nearly every gameplay mechanic in Parallax depends on interactions between the player and the environment. Because the game is built around cooperative puzzle solving and asymmetric gameplay, the PCS became responsible for a wide range of interconnected systems including movement, interaction handling, role-specific abilities, camera behavior, audio feedback, and visual feedback.

The structure and architecture of the PCS were not clearly defined during the early stages of development. Initially, much of the gameplay functionality was implemented within a single movement script, which gradually evolved into a “god script” responsible for everything from player movement to audio playback and interaction handling. As development progressed, this approach became increasingly difficult to maintain and extend.

Alternative approaches using Rigidbody-based movement were also considered during development, as discussed in community tutorials and discussions regarding Unity movement systems [4]. However, the CharacterController-based approach was ultimately chosen due to the greater control it provided over movement responsiveness, collision handling, and traversal behavior.

To address the growing complexity of the controller architecture, a more modular approach was introduced, where functionality was separated into multiple smaller and more specialized scripts with clearly defined responsibilities. Systems communicated through shared gameplay logic, interfaces, component references, and event-driven behavior, allowing the different gameplay systems to remain loosely coupled while still functioning together as part of the overall player experience.

This restructuring improved maintainability, readability, scalability, and flexibility throughout development, while also reducing dependencies between systems and making it easier to expand the PCS with additional gameplay mechanics and role-specific features.

Overall, the PCS provided the foundation required to support the intended gameplay experience in Parallax, while also allowing the project to scale with additional mechanics and gameplay features throughout development.

Bibliography

- [1] U. CharacterController, "CharacterController." [Online]. Available: <https://docs.unity3d.com/ScriptReference/CharacterController.html>
- [2] D. Gurus, "What are SOLID design principles?." [Online]. Available: <https://www.designgurus.io/answers/detail/solid-design-principles>
- [3] SynthRig, "Beginner Character controller tutorial." [Online]. Available: https://www.reddit.com/r/Unity3D/comments/1mmella/beginner_character_controller_tutorial/
- [4] K. Games, "Stop Using Unity's Character Controller!." [Online]. Available: <https://www.youtube.com/watch?v=al5Yhi6l-JI>